

AC 自动机

主体

```
int getid(char c) {
    if ('A' <= c && c <= 'Z') return c - 'A';
    if ('a' <= c && c <= 'z') return c - 'a' + 26;
    return c - '0' + 26 + 26;
}

struct AC {
    const int root = 1;
    int t[N][siz], fail[N], deg[N];
    bool vis[N];
    int cnt = root;

    void init() {
        for (int i = 1; i <= cnt; i++) memset(t[i], 0, sizeof(t[i]));
        memset(fail, 0, sizeof(int) * (cnt + 1));
        memset(vis, 0, sizeof(bool) * (cnt + 1));
        memset(deg, 0, sizeof(int) * (cnt + 1));
        cnt = root;
    }

    int insert(string &s) {
        int p = root;
        for (auto c: s) {
            int nex = getid(c);
            if (!t[p][nex]) t[p][nex] = ++cnt;
            p = t[p][nex];
        }
        return p;
    }

    void getfail() {
        queue<int> Q;
        for (int i = 0; i < siz; i++) {
            if (t[root][i]) {
                fail[t[root][i]] = root;
                Q.push(t[root][i]);
                deg[root]++;
            }
        }
    }
};
```

```

        } else t[root][i] = root;
    }
    while (Q.size()) {
        int v = Q.front();
        Q.pop();
        for (int i = 0; i < siz; i++) {
            int &u = t[v][i];
            if (u) {
                fail[u] = t[fail[v]][i];
                deg[t[fail[v]][i]]++;
                Q.push(u);
            } else u = t[fail[v]][i];
        }
    }
}

void query(string &s) {
    int p = root;
    for (auto c: s) {
        int nex = getid(c);
        p = t[p][nex];
        vis[p] = 1;
    }
}

void getans() {
    queue<int> Q;
    for (int i = 1; i <= cnt; i++) if (!deg[i]) Q.push(i);
    while (Q.size()) {
        int v = Q.front();
        Q.pop();
        deg[fail[v]]--;
        if (vis[v]) vis[fail[v]] = 1;
        if (!deg[fail[v]]) Q.push(fail[v]);
    }
}
} ac;

```

调用顺序

```
ac.insert(s);
ac.getfail();
ac.query(t);
ac.getans();
```

tarjan

强连通分量

```
int dfn[N], low[N], col[N], tot, cnt;
stack<int> S;
void tarjan(int v) {
    low[v] = dfn[v] = ++tot;
    S.push(v);
    for (auto u: E[v]) {
        if (!dfn[u]) {
            tarjan(u);
            low[v] = min(low[v], low[u]);
        } else if (!col[u]) {
            low[v] = min(low[v], dfn[u]);
        }
    }
    if (low[v] == dfn[v]) {
        col[v] = ++cnt;
        while (S.top() != v) {
            col[S.top()] = cnt;
            S.pop();
        }
        S.pop();
    }
}
```

割点

```
void dfs(int v, int fa) {
    dfn[v] = low[v] = ++tot;
    int son = 0;
    for (int i = 0; i < E[v].size(); i++) {
        int x = E[v][i];
        if (!dfn[x]) {
            son++;
            dfs(x, v);
            low[v] = min(low[v], low[x]);
            if (v != 1 && low[x] >= dfn[v]) {
                mark[v] = true;
            }
            if (v == 1 && son == 2) {
                mark[1] = true;
            }
            // mark 了的是割点
        } else if (x != fa)
            low[v] = min(low[v], dfn[x]);
    }
}

dfs(1, 1);
```

桥

```
vector<int> E[N];
int dfn[N], low[N], tot, n, m;
vector<pair<int, int>> ans;
void dfs(int v, int fa) {
    dfn[v] = low[v] = ++tot;
    for (int i = 0; i < E[v].size(); i++) {
        int x = E[v][i];
        if (!dfn[x]) {
            dfs(x, v);
            low[v] = min(low[v], low[x]);
            if (low[x] > dfn[v]) {
                ans.emplace_back(x, v);
                // 加入 ans 中的就是桥
            }
        } else if (x != fa)
            low[v] = min(low[v], dfn[x]);
    }
}

dfs(1, 1);
```

KMP

```
int nex[N];
void getnex(const string &s) {
    int n = s.size();
    int i = 0, j = -1;
    nex[0] = -1;
    while (i < n) {
        if (s[i] == s[j] || j == -1)
            nex[++i] = ++j;
        else j = nex[j];
    }
}
int kmp(const string &a, const string &b) {
    int i = 0, j = 0, res = 0;
    getnex(b);
    while (i < a.size()) {
        if (j == -1 || a[i] == b[j]) {
            i++;
            j++;
        } else j = nex[j];
        if (j == b.size()) {
            j = nex[j];
            res++;
        }
    }
    return res;
}
```

Manacher

对于一个位置 i , $[i - f_i + 1, i + f_i - 1]$ 是最长的以 i 为中心的奇回文串（对应扩充串），而 g_i 表示以 i 为起始位置的最长回文串长度（对应原串）。

$\max(f) - 1$ 是原字符串的最长回文子串长度。

g 是答案数组，表示从原串每个位置出发，最长的回文前缀长度。

t 是辅助数组，用来记录“某个起点能到的最远右边界”。

```

void manacher(std::string a, std::vector<int> &f, std::vector<int> &g) {
    int n = a.size(), m;
    std::vector<char> s(n * 2 + 3);
    for (int i = 1; i <= n; ++i) s[i << 1] = a[i - 1], s[i << 1 | 1] = '#';
    s[0] = '$', s[1] = '#', s[m = (n + 1) << 1] = '\0';

    f.resize(m + 1, 0); g.resize(n + 1, 0); f[1] = 1;
    for (int r = 0, p = 0, i = 2; i < m; ++i) {
        for (f[i] = r > i ? std::min(r - i, f[p * 2 - i]) : 1;
            s[i - f[i]] == s[i + f[i]]; ++f[i]);
        if (i + f[i] > r) r = i + f[i], p = i;
    }

    std::vector<int> t(m + 1, 0);
    auto update = [] (int &a, int b) { if (a < b) a = b; };
    for (int i = 0; i <= m; ++i) t[i] = 0;
    for (int i = 2; i < m; ++i) update(t[i - f[i] + 1], i + 1);
    for (int i = 1; i < m; ++i) update(t[i], t[i - 1]);
    for (int i = 2; i < m; ++i) g[i >> 1] = t[i] - i;
}

```

欧拉筛

```

std::vector<int> prime;
bool notprime[N] = {1, 1};
int minp[N];
void Euler(int n) {
    for (int i = 2; i <= n; i++) {
        if (!notprime[i]) {
            prime.push_back(i);
            minp[i] = i;
        }
        for (int p : prime) {
            if (i * p > n) break;
            notprime[i * p] = true;
            minp[i * p] = p;
            if (i % p == 0) break;
        }
    }
}

```

O1 LCA

```
void dfs(int u, int fa, int d) {
    first[u] = (int)euler.size();
    euler.push_back(u);
    depEuler.push_back(d);

    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u, d + 1);
        euler.push_back(u);
        depEuler.push_back(d);
    }
}

int better(int x, int y) {
    // x,y 是 euler 序中的位置，返回深度更小的位置
    return depEuler[x] < depEuler[y] ? x : y;
}

void build_st() {
    int m = euler.size();
    for (int i = 0; i < m; i++) st[i][0] = i;

    for (int j = 1; (1 << j) <= m; j++) {
        for (int i = 0; i + (1 << j) - 1 < m; i++) {
            st[i][j] = better(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
        }
    }

    lg2_[1] = 0;
    for (int i = 2; i <= m; i++) lg2_[i] = lg2_[i >> 1] + 1;
}

int lca(int u, int v) {
    int l = first[u], r = first[v];
    if (l > r) swap(l, r);
    int k = lg2_[r - l + 1];
    int pos = better(st[l][k], st[r - (1 << k) + 1][k]);
    return euler[pos];
}
```


dynamic_bitset

```
struct Bitset {
    using ull = unsigned long long;
    static const int W = 64;

    int n;                // 总位数
    std::vector<ull> a;    // 存储

    Bitset(int _n = 0) { init(_n); }

    void init(int _n) {
        n = _n;
        a.assign((n + W - 1) / W, 0);
    }

    // ===== 基本操作 =====
    void set(int pos) {
        a[pos / W] |= (1ull << (pos % W));
    }

    void reset(int pos) {
        a[pos / W] &= ~(1ull << (pos % W));
    }

    void flip(int pos) {
        a[pos / W] ^= (1ull << (pos % W));
    }

    bool test(int pos) const {
        return (a[pos / W] >> (pos % W)) & 1;
    }

    // ===== 批量操作 =====
    void set_all() {
        for (auto &x : a) x = ~0ull;
        trim();
    }

    void reset_all() {
        for (auto &x : a) x = 0;
    }
}
```

```

void flip_all() {
    for (auto &x : a) x = ~x;
    trim();
}

// 防止多余位污染
void trim() {
    int r = n % W;
    if (r) a.back() &= ((1ull << r) - 1);
}

// ===== 统计 =====
int count() const {
    int res = 0;
    for (auto x : a) res += __builtin_popcountll(x);
    return res;
}

bool any() const {
    for (auto x : a) if (x) return true;
    return false;
}

bool none() const {
    return !any();
}

// ===== 位运算 =====
Bitset operator&(const Bitset &o) const {
    Bitset res(n);
    for (int i = 0; i < a.size(); i++)
        res.a[i] = a[i] & o.a[i];
    return res;
}

Bitset operator|(const Bitset &o) const {
    Bitset res(n);
    for (int i = 0; i < a.size(); i++)
        res.a[i] = a[i] | o.a[i];
    return res;
}

```

```

Bitset operator^(const Bitset &o) const {
    Bitset res(n);
    for (int i = 0; i < a.size(); i++)
        res.a[i] = a[i] ^ o.a[i];
    return res;
}

```

```

Bitset operator~() const {
    Bitset res(n);
    for (int i = 0; i < a.size(); i++)
        res.a[i] = ~a[i];
    res.trim();
    return res;
}

```

// ===== 左移 =====

```

Bitset operator<<(int k) const {
    Bitset res(n);
    int shift_block = k / W;
    int shift_bit = k % W;

    for (int i = a.size() - 1; i >= 0; i--) {
        if (i - shift_block < 0) continue;
        res.a[i] |= a[i - shift_block] << shift_bit;
        if (shift_bit && i - shift_block - 1 >= 0) {
            res.a[i] |= a[i - shift_block - 1] >> (W - shift_bit);
        }
    }
    res.trim();
    return res;
}

```

// ===== 右移 =====

```

Bitset operator>>(int k) const {
    Bitset res(n);
    int shift_block = k / W;
    int shift_bit = k % W;

    for (int i = 0; i < a.size(); i++) {
        if (i + shift_block >= a.size()) continue;
        res.a[i] |= a[i + shift_block] >> shift_bit;
        if (shift_bit && i + shift_block + 1 < a.size()) {
            res.a[i] |= a[i + shift_block + 1] << (W - shift_bit);
        }
    }
    res.trim();
    return res;
}

```

```

    }
}
res.trim();
return res;
}
int find_first_one() const {
    for (int i = 0; i < a.size(); i++) {
        if (a[i]) {
            return i * W + __builtin_ctzll(a[i]);
        }
    }
    return -1;
}
int find_first_zero() const {
    for (int i = 0; i < a.size(); i++) {
        ull x = ~a[i];
        if (i == (int)a.size() - 1) {
            int r = n % W;
            if (r) x &= ((1ull << r) - 1);
        }
        if (x) {
            return i * W + __builtin_ctzll(x);
        }
    }
    return -1;
}
};

```

基础设置

```

std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
std::cout.tie(nullptr);
std::cout << std::fixed << std::setprecision(10);

```

pbds 库

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
namespace pbds = __gnu_pbds;
template<typename T>
using rb_tree = pbds::tree<
    T,
    pbds::null_type,
    std::less<T>,
    pbds::rb_tree_tag,
    pbds::tree_order_statistics_node_update
>;
int main() {
    rb_tree<int> s;
    // 插入元素
    s.insert(5);
    // 删除元素
    s.erase(5);
    // 表示有多少元素 < 7 => 即 7 的排名
    int rank7 = s.order_of_key(7);
    // 找某个元素的排名（如果不存在则返回它应该插入的位置）
    int rank = s.order_of_key(10);
    // 取排名为 0 的元素（从 0 开始计）
    auto it = s.find_by_order(0);
    // 遍历：按排序后的顺序
    for (auto x : s) std::cout << " " << x;
    // 如果需要判断是否存在，可以检查：s.find(x) != s.end()
    return 0;
}
```